

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**
Aula: 08 - Introdução a algoritmos de otimização

Professor: **Raphael Martins Brum**

2026

Introdução à otimização

- **Objetivo:** encontrar a melhor solução para um problema
- **Problemas** otimizáveis e suas **soluções:**
 - Rota de menor custo: endereço 1 -> endereço 2 -> endereço 3 -> ... -> endereço 5
 - Máximo lucro em carteira de investimentos: 20% no Ativo 1, 80% no Ativo 2
 - Alocação de tarefas: Tarefa 1 no PC 4, Tarefa 2 no PC 2, Tarefa 3 no PC 5
- Otimizar $\Leftrightarrow$ Minimizar ou maximizar alguma métrica



Introdução à otimização

- Espaço de busca: conjunto de todas as possíveis soluções para um problema (contínuo x discreto)



Problema do Troco x Algoritmo Guloso (*greedy*)

- Constrói uma solução a partir de escolhas localmente ótimas
- Considere o problema do troco:
 - Precisamos fornecer o troco para uma compra, e possuímos um conjunto de moedas para compô-lo
 - Precisamos “cobrir” o troco com as moedas que temos
 - Queremos minimizar o número de moedas entregues
- No algoritmo guloso, vamos adotar a estratégia de utilizar sempre a **maior moeda primeiro**
- Isso leva ao ótimo em qualquer caso?



Problema do Troco x Algoritmo Guloso

```
def troco_guloso(valor, moedas):  
    moedas_sorted = sorted(moedas,  
                           reverse=True)  
    resultado = []  
    for m in moedas_sorted:  
        while valor >= m:  
            resultado.append(m)  
            valor -= m  
            valor = round(valor, 2)  
    return resultado  
  
moedas = [25, 25, 10, 5, 1, 1, 1,  
          1, 10]  
print(troco_guloso(67, moedas))
```

Saída:

[25, 25, 10, 5, 1, 1]

O resultado é sempre o ótimo, pois o sistema de moedas do Real é **canônico**.



Problema da Mochila

- Considere uma mochila que possui um limite de peso
- Considere itens com duas propriedades: peso e valor
 - Considere que os itens podem ser fracionados (queijo?)
- Queremos levar o conjunto mais valioso, respeitando a capacidade
- **Conceitos:**
 - Função objetivo: maximizar o valor financeiro
 - Restrição (*constraint*): respeitar a capacidade máxima da mochila



Problema da Mochila x Algoritmo Guloso

```
def mochila(cap, itens):
    itens_ord = sorted(itens,
        key=lambda x: x[1] / x[2],
        reverse=True)
    total = 0.0
    sobra = cap
    escolhidos = []

    for item in itens_ord:
        if sobra <= 0:
            break
        p, v = item[1], item[2]
        fracao = min(1.0, sobra / p)
        ganho = v * fracao
        total += ganho
        sobra -= p * fracao
        escolhidos.append([item[0], fracao*p, ganho])

    return [total, cap-sobra, sobra], escolhidos
```

```
itens = [{"A", 10, 60}, {"B", 20, 100}, {"C", 30, 120}]
capacidade = 50

print(mochila(capacidade, itens))
```

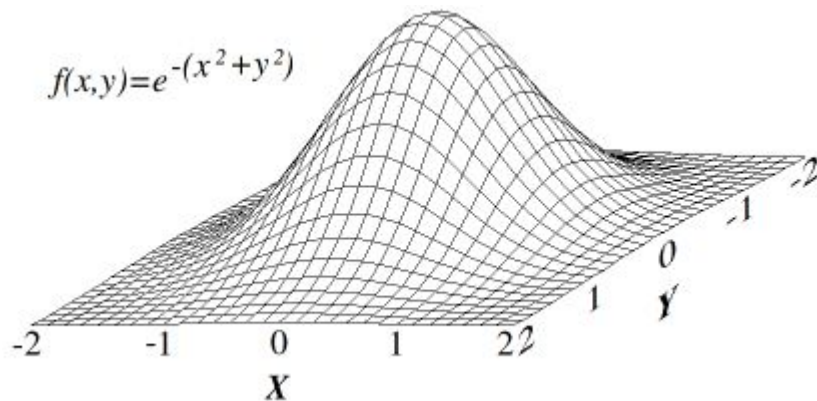
Distribuição atingida:

30kg do item C (R\$ 120)
20kg do item B (R\$ 100)
Total = R\$ 220



Hill Climbing

- Começa-se com uma solução arbitrária
- Move-se (altera-se a solução) apenas numa direção que a melhore
- HC é ótimo apenas para problemas **convexos**



Exemplo de *Hill Climbing*

```
import random, math

def vizinhos(x, delta=0.1):
    return [x + delta, x - delta]

def f(x): # minimizar
    return -(math.sin(x)*x)

def hill_climbing(iters=1000, x_inicial=None):
    x = x_inicial if x_inicial is not None else
        random.uniform(0, 10)
    for _ in range(iters):
        melhor_viz = min(
            vizinhos(x), key=f)
        if f(melhor_viz) < f(x):
            x = melhor_viz
        else:
            break
    return x, f(x)

sol, val = hill_climbing()
print(f"x={sol:.3f}, f={val:.3f}")
```

No exemplo ao lado, vamos encontrar o mínimo de $f(x) = -\sin(x)/x$

O ótimo global:

$x = 7,98$, $f(x) = -7,92$

Tente os seguintes chutes iniciais:

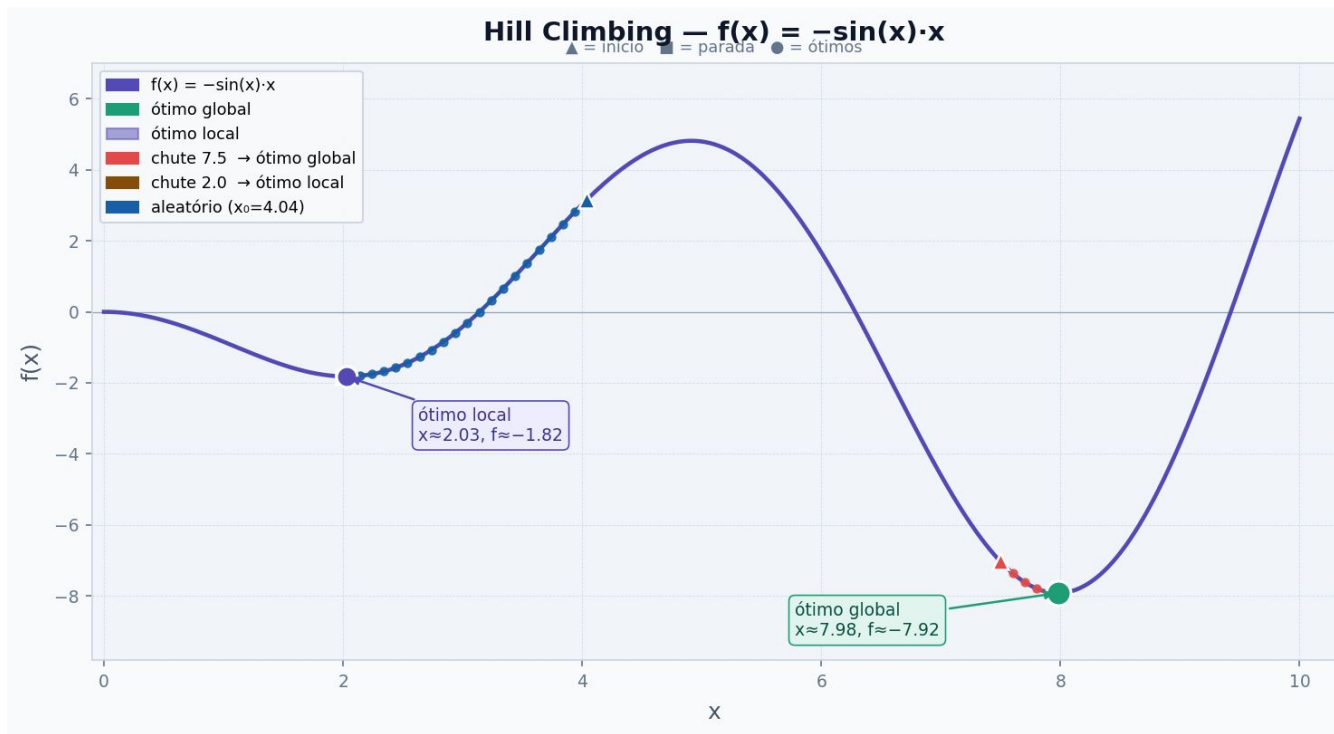
$x = 7,5$

$x = 2$

Mínimo global x Mínimo local



Hill Climbing e os mínimos locais



Hill Climbing x Problema da Mochila

```
def hc_mochila(itens, cap, max_iter=5_000, delta=0.05,
               seed=42):

    random.seed(seed)
    n = len(itens)

    atual = [random.random() for _ in range(n)]
    melhor_val = avaliar(atual, itens, cap)

    historico = [melhor_val]

    for _ in range(max_iter):
        viz = gerar_vizinho(atual, delta)
        v_viz = avaliar(viz, itens, cap)
        if v_viz > melhor_val:
            atual, melhor_val = viz, v_viz
            historico.append(melhor_val)

    escolhidos = [[it[0], f * it[1], f * it[2]]
                  for f, it in zip(atual, itens) if f > 1e-6 ]
    return round(melhor_val, 2), escolhidos, historico
```

avaliar() devolve o valor (R\$) de uma dada solução

gerar_vizinho() gera uma solução similar à solução fornecida



Hill Climbing x Problema da Mochila

```
import random

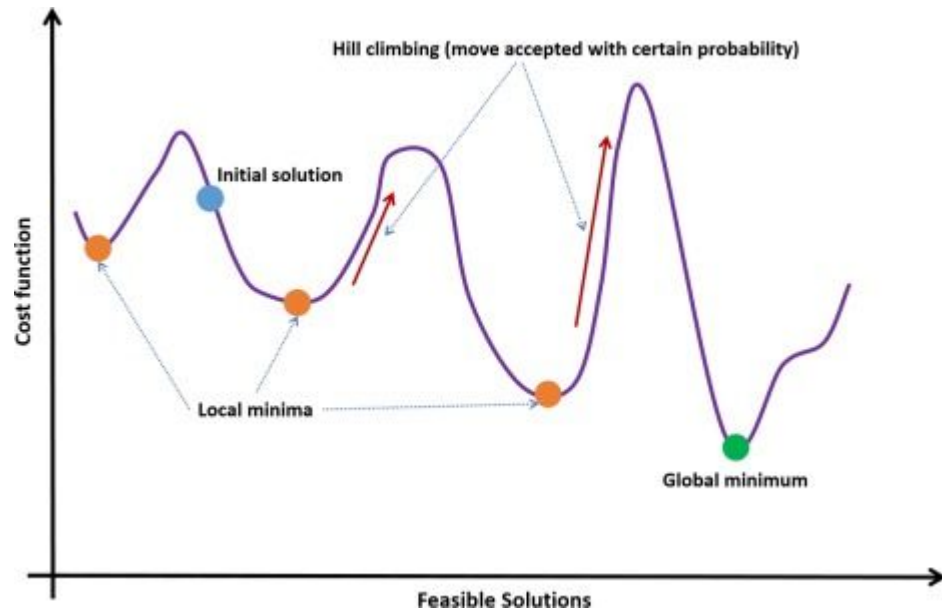
def avaliar(fracoas, itens, cap):
    peso = sum(f * it[1] for f, it in zip(fracoas, itens))
    valor = sum(f * it[2] for f, it in zip(fracoas, itens))
    if peso > cap + 1e-9:
        return -float("inf")
    return valor

def gerar_vizinho(fracoas, delta=0.05):
    viz = fracoas[:]
    i = random.randrange(len(viz))
    viz[i] = max(0.0, min(1.0, viz[i] + random.uniform(-delta, delta)))
    return viz
```



Recozimento simulado ou *Simulated Annealing* (SA)

- Inspirado nos processos de recozimento metalúrgicos
- Diferentemente do HC, move-se para soluções piores
- De início, a “agitação térmica” é permissiva com a piora
- Após, a temperatura “diminui” e essa permissividade vai gradualmente se extinguindo
 - Probabilidade: $\exp(-\Delta/T)$
 - Esfriamento: $T = \alpha T$, $0 < \alpha < 1$



Problema do Caixeiro Viajante (TSP)

- *Travelling Salesman Problem*
- Dada uma lista de cidades e as distâncias entre essas cidades, qual é a rota que minimiza o trajeto e visita todas as cidades uma única vez?
- Problema clássico de otimização combinatória, NP-difícil



Simulated Annealing x TSP

```
def simulated_annealing(cidades, T=1000, alpha=0.995, T_min=0.01):
    rota = list(range(len(cidades)))
    random.shuffle(rota)
    custo = distancia_total(rota, cidades)
    melhor, melhor_custo = rota[:], custo

    while T > T_min:
        nova_rota = vizinho_troca(rota)
        novo_custo = distancia_total(nova_rota, cidades)
        delta = novo_custo - custo

        if delta < 0 or random.random() < math.exp(-delta / T):
            rota, custo = nova_rota, novo_custo
            if custo < melhor_custo:
                melhor, melhor_custo = rota[:], custo
        T *= alpha

    return melhor, melhor_custo
```



Simulated Annealing x TSP

```
def distancia_total(rota, cidades):
    total = 0
    for i in range(len(rota)):
        a, b = cidades[rota[i]], cidades[rota[(i+1) %
len(rota)]]
        total += math.dist(a, b)
    return total

def vizinho_troca(rota):
    nova = rota[:]
    i, j = random.sample(range(len(rota)), 2)
    nova[i], nova[j] = nova[j], nova[i]
    return nova

cidades = {i: (random.uniform(0,100),
random.uniform(0,100)) for i in range(20)}
rota, dist = simulated_annealing(cidades)
print(f"Melhor distância: {dist:.2f}")
```



Simulated Annealing x Problema da Mochila

```
def simulated_annealing(  
    T=100.0,  
    alpha=0.995,  
    T_min=0.001,  
    sol_inicial=None,  
    seed=42):  
  
    random.seed(seed)  
    n = len(itens)  
  
    sol = sol_inicial if sol_inicial  
        is not None else [0.0] * n  
    custo = avaliar(sol)  
  
    melhor, melhor_custo =  
        sol[:], custo  
    historico = [custo]
```

```
    while T > T_min:  
        viz = vizinho(sol)  
        delta = avaliar(viz) - custo  
  
        if delta > 0 or random.random() < math.exp(delta / T):  
            sol, custo = viz, avaliar(viz)  
  
        if custo > melhor_custo:  
            melhor, melhor_custo = sol[:], custo  
  
        T *= alpha  
        historico.append(melhor_custo)  
  
    return melhor, round(melhor_custo, 4), historico
```



Simulated Annealing x Problema da Mochila

```
def avaliar(fracoas):
    peso = sum(f * it[1] for f, it in
        zip(fracoas, itens))
    valor = sum(f * it[2] for f, it in
        zip(fracoas, itens))
    return valor if peso <= CAPACIDADE + 1e-9
        else -float("inf")
```

```
def vizinho(fracoas):
    viz = fracoas[:]
    i = random.randrange(len(viz))
    viz[i] = max(0.0, min(1.0, viz[i] +
        random.uniform(-0.2, 0.2)))
    peso = sum(f * it[1] for f, it in
        zip(viz, itens))
```

```
if peso > CAPACIDADE:
    excesso = peso - CAPACIDADE
    ordem = sorted(range(len(itens)),
        key=lambda k: itens[k][2] / itens[k][1])
    for k in ordem:
        reducao = min(viz[k] * itens[k][1], excesso)
        viz[k] -= reducao / itens[k][1]
        excesso -= reducao
    if excesso <= 1e-9: break
return viz
```



Threshold Accept

- Similar ao SA, mas substitui a probabilidade por um limiar determinado
 - Variação do vizinho $< \theta$? Então aceita.
 - Reduz o próprio limiar: $\theta = \alpha\theta$, $0 < \alpha < 1$



Projeto 9: TSP com diferentes algoritmos

Faça sua própria implementação dos algoritmos de otimização vistos aqui para resolver os **benchmarks** de TSP da TSPLIB95, encontráveis em <https://github.com/mastqe/tsplib>.

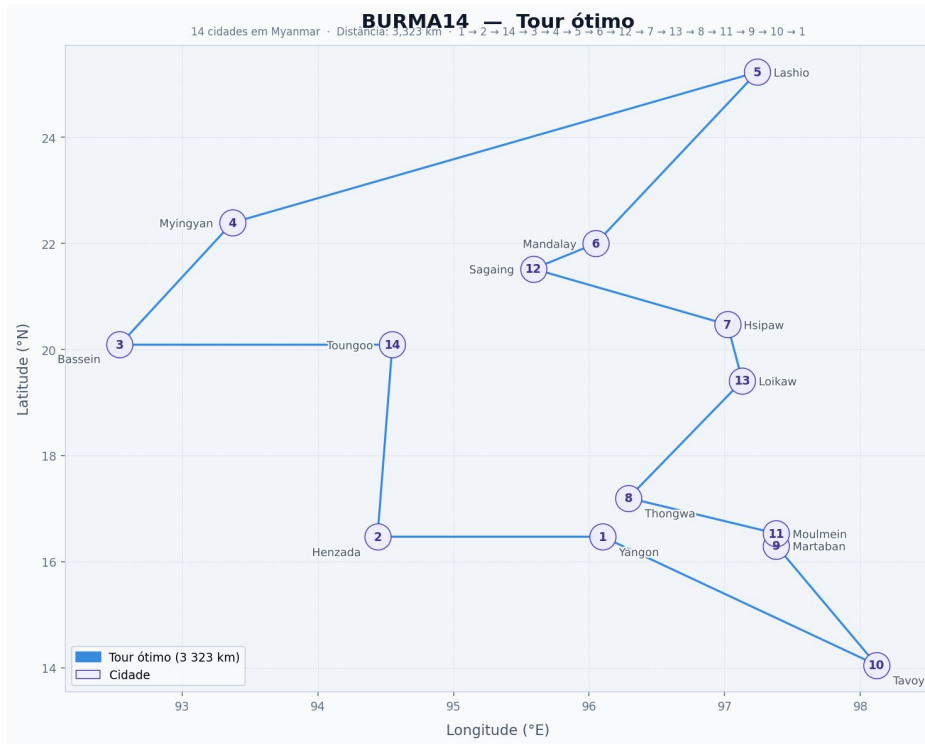
Registre os resultados para cada benchmark em uma tabela.

Utilize o código abaixo para fazer a leitura dos benchmarks.

```
# pip install tsplib95 (alternativa)
# PARSE
import tsplib95
prob = tsplib95.load('berlin52.tsp')
nos = list(prob.get_nodes())
```



TSPLIB - Problema “burma14”



NAME: burma14
TYPE: TSP
COMMENT: 14-Staedte in Burma (Zaw Win)
DIMENSION: 14
EDGE_WEIGHT_TYPE: GEO
EDGE_WEIGHT_FORMAT: FUNCTION
DISPLAY_DATA_TYPE: COORD_DISPLAY
NODE_COORD_SECTION

1	16.47	96.10
2	16.47	94.44
3	20.09	92.54
4	22.39	93.37
5	25.23	97.24
6	22.00	96.05
7	20.47	97.02
8	17.20	96.29
9	16.30	97.38
10	14.05	98.12
11	16.53	97.38
12	21.52	95.59
13	19.41	97.13
14	20.09	94.55

EOF

